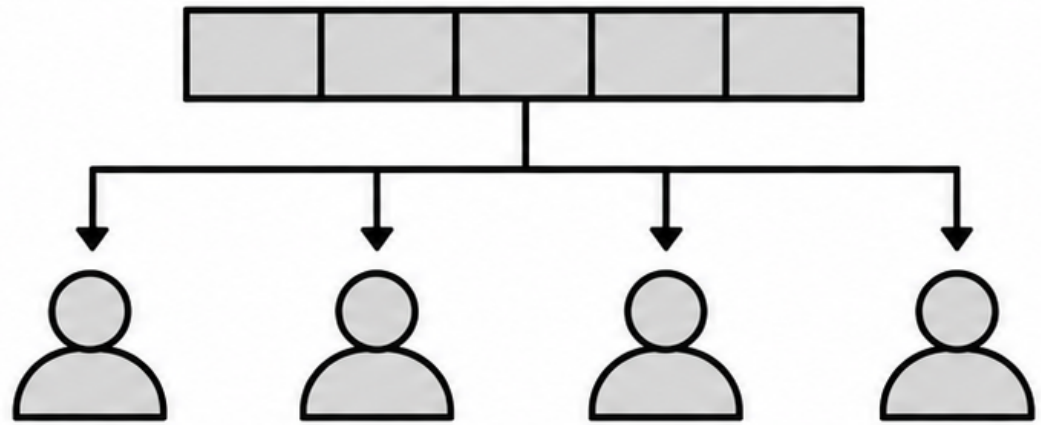
Choosing how to work also takes work

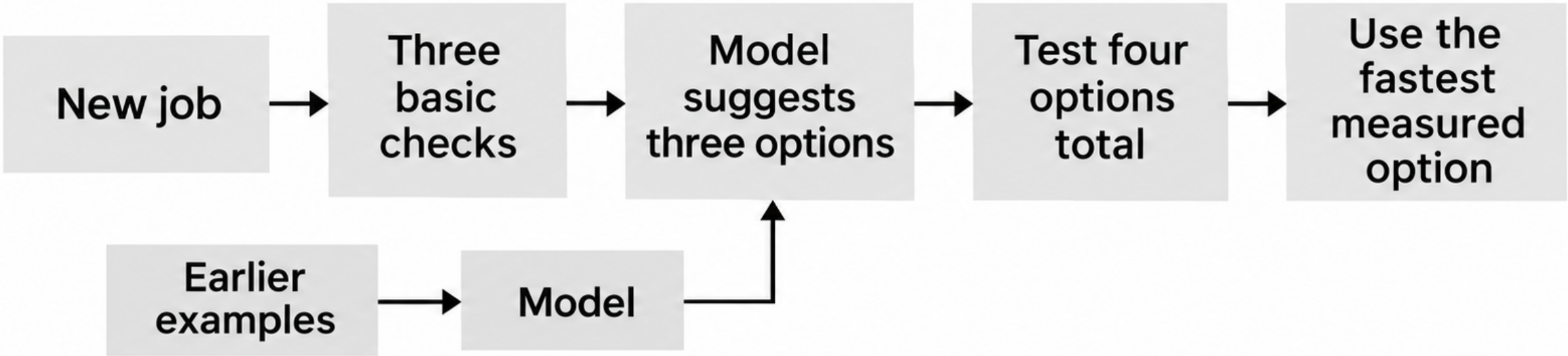
- CPU threads are workers. A scheduler divides the work between them.
- There are 12 scheduling options to choose from.
- Testing every option can cost more time than it saves.



Our question: can we make the choice more cheaply?

Our workflow in one picture

- Learn from earlier jobs.
- Test a short list on the new job.
- Pick one option and keep using it.



The basic STATIC option is included in the four.

What earlier research gives us

- LB4OMP (2022): a library with several ways to divide loop work.
- Auto4OMP (2022): automatically choose a scheduler and its batch size.
- Comparative study (2025): compares selection strategies and examines exploration cost.

We reuse existing schedulers and study the cost of choosing.

Our goal is to spend less on choosing

- Try fewer scheduling options.
- Measure both elapsed time and processor time.
- Check whether the final choice is still fast enough.

Success means a cheaper overall job, with an acceptable speed trade-off.

The three jobs we tested

- Mandelbrot: calculate image pixels; some pixels take longer.
- Array arithmetic: repeat the same simple calculation across a large list.
- Triangle counting: find groups of three connected graph points.

$$a_i = b_i + 3c_i$$

Different jobs create different work-sharing problems.

The machine and experiment

- AMD Ryzen 5 7235HS; four physical CPU cores; WSL2 Linux.
- Tested with two and four threads using C++ and OpenMP.
- 24 test scenarios, seven methods, five repeated blocks: 840 main runs.

One machine and three workload families: the conclusions have that scope.

What we compared

- Full 12: test every option three times — 36 test executions.
- Guided 4: test four options three times — 12 test executions.
- Also check smaller/larger budgets, prediction only, random choices, and one fixed choice.

A test execution means running the loop once to measure it.

First, learn from earlier jobs

- Measure every scheduler on separate training inputs.
- Learn which job characteristics suggest a good option.
- Freeze the model before measuring the new test jobs.

The new test results were not training answers.

Describe the new job with six numbers

- How big is the job, and how many threads are available?
- How long do three basic STATIC checks take?
- Do some threads finish much earlier than others?

$$x = [1, \log_2 N, \log_2 P, \log_2 T_0, L_0, V_0]^T$$

x is just an information card for the job.

Turn speed into an easy training score

- Faster than STATIC gets a positive score.
- Slower than STATIC gets a negative score.
- Extreme scores are capped between −1 and +1.

$$r = \mathrm{clip}\left(\frac{-\log(T/T_{\mathrm{STATIC}})}{\log 4}, -1, 1\right)$$

Twice as fast: +0.5. Same speed: 0. Twice as slow: −0.5.

Learn a simple prediction rule

- Fit one small scoring rule for each scheduler.
- Each rule learns how much the six job features matter.
- A stabilizing penalty prevents extreme coefficients.

$$A = \lambda I + w \sum x x^T$$

$$b = w \sum x r$$

$$\theta = A^{-1} b$$

This is ridge regression: a weighted sum with a penalty.

Predict, then verify the shortlist

- Multiply the job features by the learned weights.
- Rank the options by their predicted scores.
- Measure the shortlist; actual timings decide the winner.

$$\text{predicted score} = x^T \theta$$

The model suggests. Measurements choose.

Evaluate the total cost

- Include the cost of preparing the model.
- Add the time spent choosing for this job.
- Add the time spent doing useful work.

$$\text{Total cost} = \frac{\text{training cost}}{\text{reuses}} + \text{choosing cost} + \text{useful execution cost}$$

$$J(H, R) = \frac{C_{prior}}{R} + S + \sum_{t=1}^{H} T_t$$

Stopwatch time and CPU time are measured separately.

Check how good the final choice is

- Independently measure all 12 options as an evaluation reference.
- Compare the chosen scheduler’s execution time with the fastest reference median.
- Keep this separate from the cost of searching.

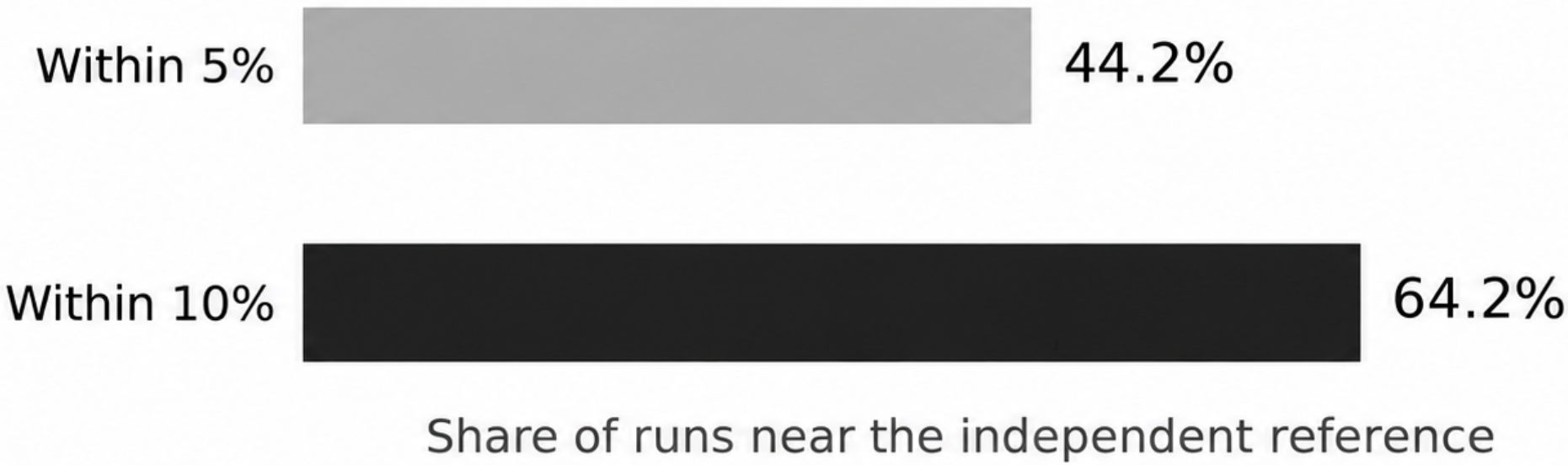
$$slowdown = \frac{\text{chosen time}}{\text{reference time}} - 1$$

$$retained\ speed = \frac{1}{1 + slowdown}$$

11 ms instead of 10 ms = 10% more time.

The model’s choices were not always near-best

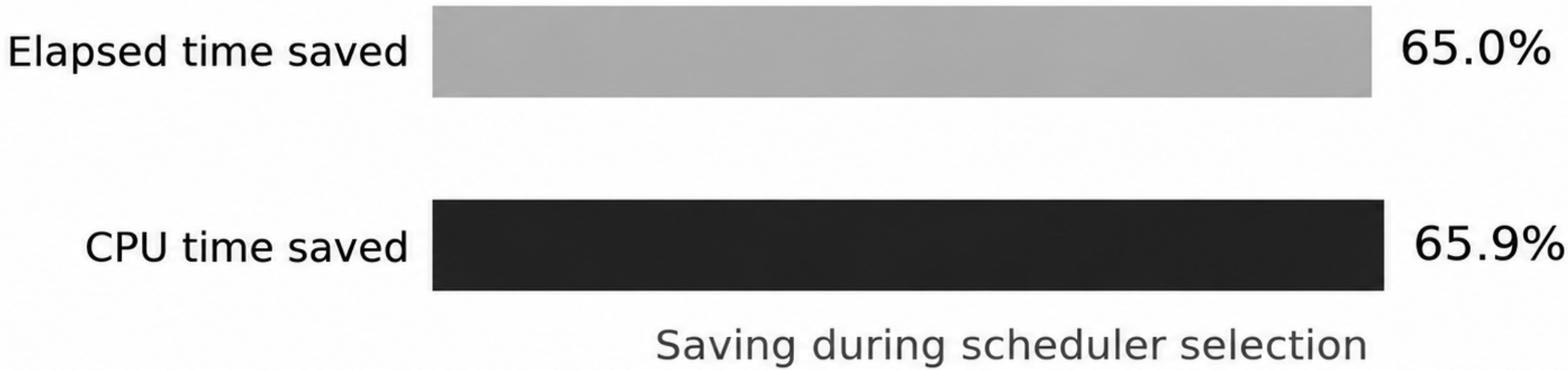
- Guided 4: 10.2% aggregate execution-time penalty to the independent reference.
- Within 5% of that reference in 44% of runs; within 10% in 64%.
- Compared with Full 12’s actual choice, no clear extra execution-time penalty.



The independent reference and Full 12’s selected option are different comparisons.

Choosing took about two-thirds less work

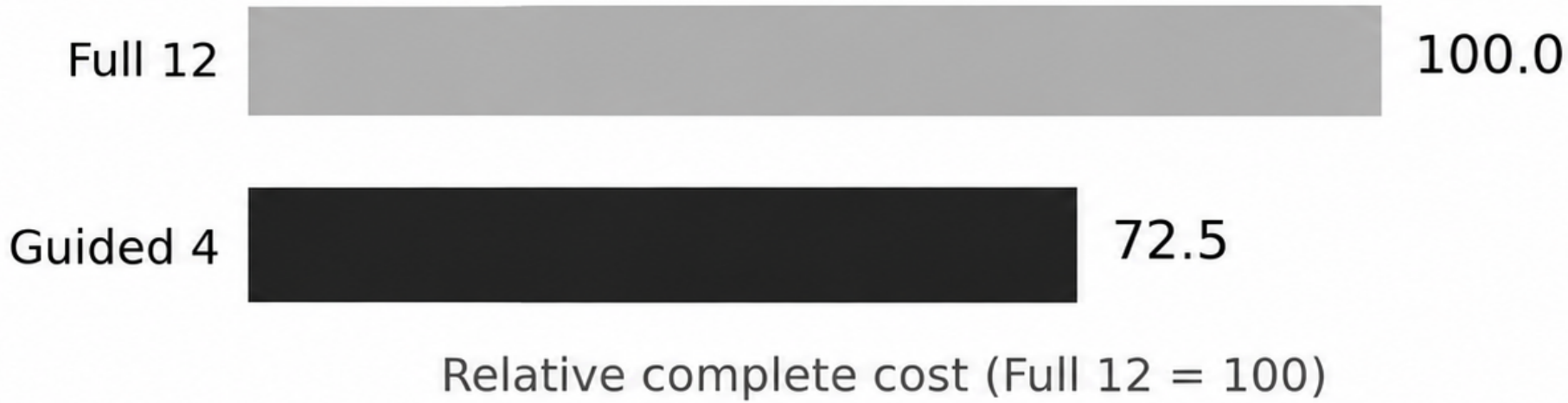
- Guided 4 used 12 test executions instead of 36.
- Choosing took 65.0% less elapsed time.
- Choosing consumed 65.9% fewer CPU seconds.



These savings are for choosing, not for the whole application.

For short jobs, the complete process was cheaper

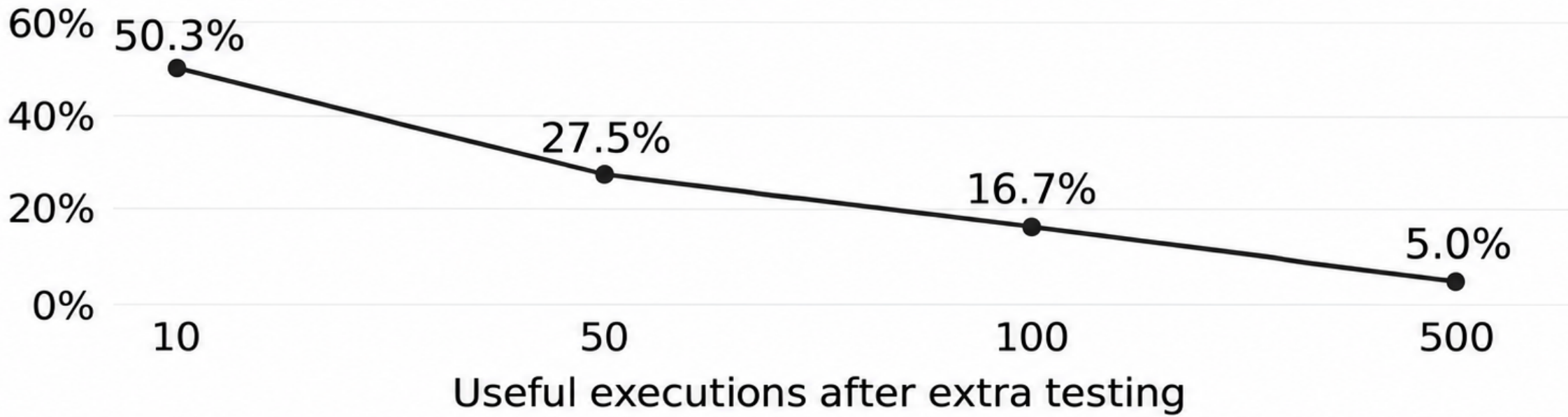
- After additional testing, every method completed the same 50 useful executions.
- Guided 4 used about 27.5% less total elapsed time.
- It also used about 27.8% fewer total CPU seconds.



Relative cost index, not seconds. Model already available; tests are extra work.

Longer jobs dilute the upfront saving

- Choosing is a one-time expense.
- Its share of the total shrinks as useful work grows.
- Any slower execution can eventually use up the saving.



$$\text{crossover} = \frac{\text{choosing-time saving}}{\text{extra time per useful execution}}$$

Preparing the model must also pay off

- Model preparation cost about 36 elapsed seconds and 84 CPU seconds.
- At 50 useful executions per job, the estimated payback is about 488 reuses for elapsed time.
- A new model for one tiny job is not economical here.

$$\text{reuses needed} \approx \frac{\text{preparation cost}}{\text{saving per job}}$$

Measured workload mixture; CPU-time payback was about 409 reuses.

The saving depends on what the tests count as

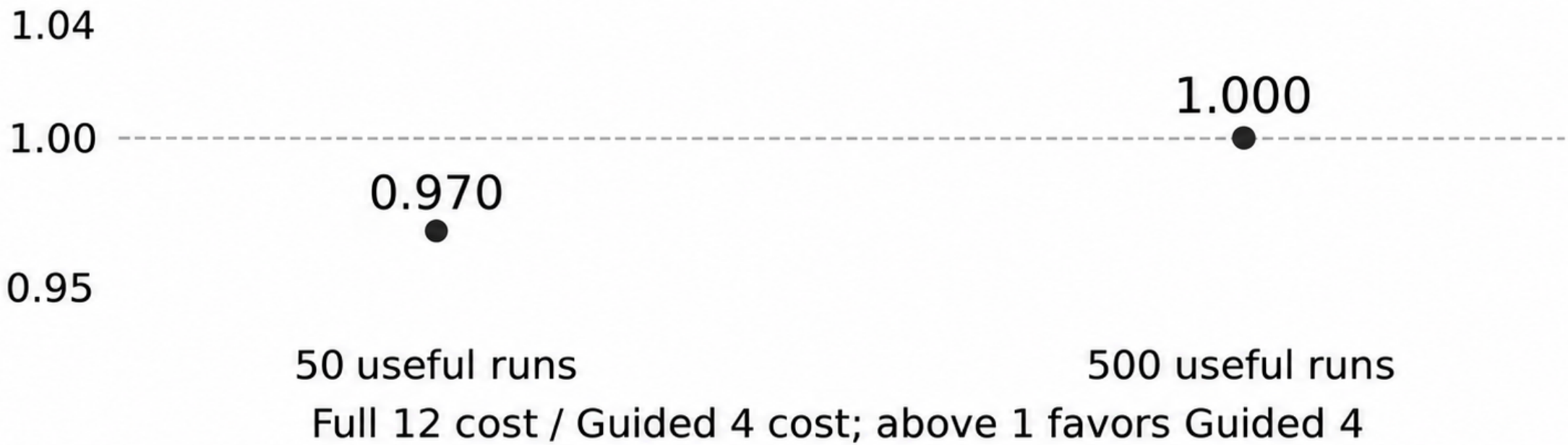
- Extra tests: Full 12 runs 36 + 50 times; Guided 4 runs 12 + 50.
- Tests already doing useful work: both methods must finish 50 total useful runs.
- In that second setting, fewer tests do not remove required work.

	Full 12	Guided 4
Extra testing	86 executions	62 executions
Testing is useful work	50 executions	50 executions

Always compare the same amount of **useful work**.

When tests were useful work, we found no saving

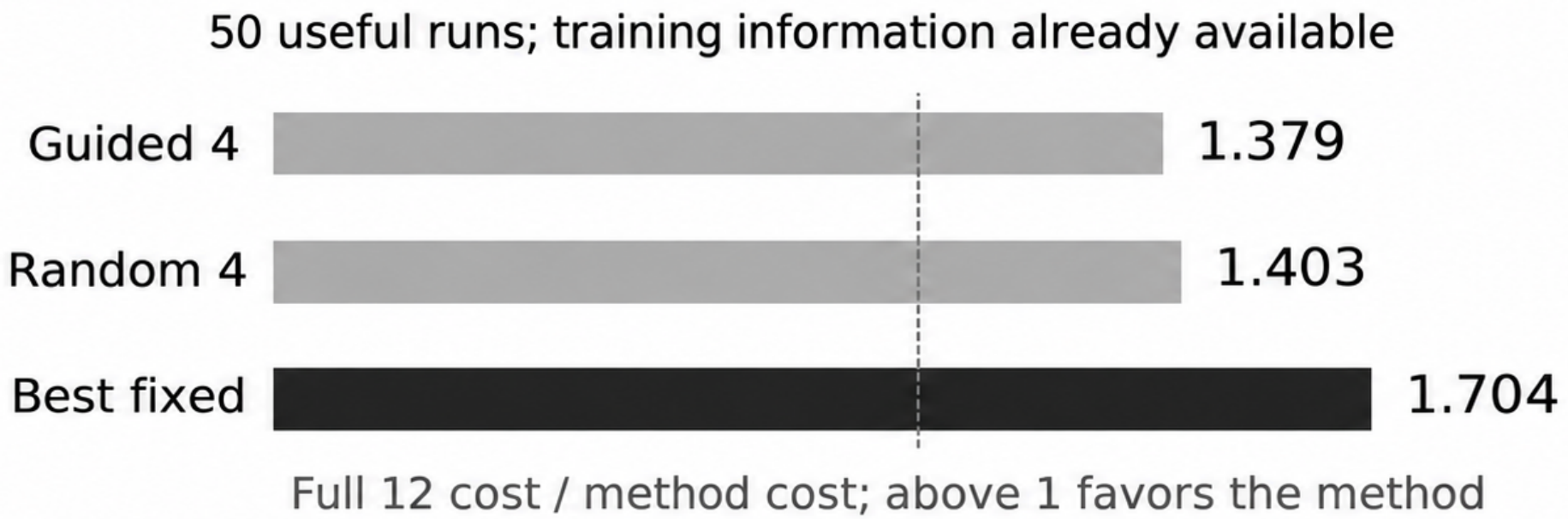
- We measured this separately using equal total useful work.
- At 50 useful runs, Guided 4 was about 3.1% slower in the point estimate.
- At 500 useful runs, the methods were essentially tied.



No demonstrated overall time-and-CPU benefit in this mode.

The simplest alternatives also worked well

- Randomly testing four options also saved substantial search work.
- Reusing one training-selected scheduler was competitive.
- These results do not establish that model ranking is necessary.



Above 1 means cheaper than Full 12. These are finite measured comparisons.

When is the trade-off worth it?

- The tests must be additional work that can genuinely be avoided.
- The model must already exist or be reused enough to repay its cost.
- The chosen scheduler’s execution quality must be acceptable.

time saved choosing > extra execution time + share of preparation cost

One machine, three workload families; no guarantee for every new program.

Conclusion

- Testing four options cut choosing time and CPU use by about two-thirds.
- With extra tests and an available model, short jobs used about 28% less total time.
- No clear saving when tests already finished useful work; simpler choices were competitive too.

A cheaper choice is useful only when the total job becomes cheaper.